

YAMBA — Guide de configuration complet

Yamba — documentation générée le 06/09/2026 depuis docs/livrables/05-YAMBA-
CONFIGURATION.md

Table des matières

Sommaire

1. Comment la configuration est organisée
 - Un piège : Nx lit aussi un .env par projet
 - Ce que fait Yamba quand une variable manque
2. Démarrage local, de zéro à une plateforme qui tourne
 - Le strict minimum
 - La séquence complète
 - Tester depuis un téléphone sur le réseau local
3. Tableau complet des variables
 - Indispensables
 - Paielement
 - Email
 - Images
 - Chiffrement
 - Connexion Google et cartes
 - Back-office
 - Événements et tâches planifiées
 - Observabilité et maintenance
 - Front
4. Les services externes, un par un
 - MongoDB Atlas
 - Redis
 - Redpanda
 - Stripe
 - ImageKit
 - Google Maps
 - Connexion Google
 - PostHog
 - Resend
 - Sentry
 - Moniteur externe
5. Générer les secrets
6. Vérifier que tout fonctionne
7. État actuel de votre installation
8. Avant la production
9. Préparer le poste pour le chantier mobile (D36, D73)

Toutes les variables d'environnement, tous les services externes, et la marche à suivre pour chacun. État du code au 06/09/2026. Ce document remplace la lecture de `.env.example` : il dit **qui lit quoi**, **ce qui se passe sans**, et **comment obtenir la valeur**.

Sommaire

1. Comment la configuration est organisée
2. Démarrage local, de zéro à une plateforme qui tourne
3. Tableau complet des variables
4. Les services externes, un par un
5. Générer les secrets
6. Vérifier que tout fonctionne
7. État actuel de votre installation
8. Avant la production
9. Préparer le poste pour le chantier mobile

1. Comment la configuration est organisée

Deux fichiers, jamais versionnés, et une règle simple pour savoir lequel utiliser.

Fichier	Lu par	Contient
<code>.env</code> à la racine	Les six services Node (gateway, auth, trip, deal, notification, message), les scripts de seed	Tout ce qui est secret : bases, clés privées, secrets de signature
<code>apps/user-ui/.env.local</code>	Le site public, au moment de la construction	Uniquement des valeurs publiques , toutes préfixées <code>NEXT_PUBLIC_</code>
<code>apps/admin-ui/.env.local</code>	Le back-office	Idem, très peu de variables

La règle : une variable préfixée `NEXT_PUBLIC_` finit dans le code JavaScript envoyé au navigateur. N'y mettez jamais une clé privée. Une clé « publiable » ou « publishable » chez un prestataire est faite pour cela ; une clé « secrète » ne l'est pas.

Conséquence pratique : changer une variable `NEXT_PUBLIC_` demande de **reconstruire** le front. Un simple rechargement de page ne suffit pas. Coupez `npm run dev` et relancez-le.

Un piège : Nx lit aussi un `.env` par projet

Nx charge les variables depuis **deux** endroits : le `.env` de la racine, et un éventuel `.env` placé dans le dossier d'un service, par exemple `apps/trip-service/.env`. Les deux sont fusionnés au démarrage.

C'est commode et c'est un piège. Une clé posée uniquement dans le dossier d'un service **fonctionne pour ce service et pour lui seul**. Les autres ne la voient pas, et rien ne le signale : le service concerné marche, un autre échoue en silence.

Le cas s'est produit avec ImageKit. Les trois clés vivaient dans `apps/trip-service/.env` : le téléversement d'une photo de colis fonctionnait, mais la **suppression** d'un avatar remplacé ou des justificatifs d'un compte effacé, qui appartient à auth-service, ne fonctionnait pas. Deux autres chemins étaient également aveugles : un bundle lancé directement avec `node --env-file=../../.env`, et le script `scripts/smoke-services.sh`.

La règle à retenir : toute variable de service va dans le `.env` de la racine, même si un seul service la lit aujourd'hui. Un `.env` dans un dossier de service est un vestige à supprimer, jamais un endroit où ajouter quelque chose.

Pour vérifier qu'aucun fichier ne traîne :

```
ls apps/*.env apps/*.env.local 2>/dev/null # ne doivent rester que les .env.local des deux fronts
git ls-files | grep -E "\.env" | grep -v example # doit être vide : aucun secret versionné
```

Un même identifiant peut apparaître des deux côtés. C'est le cas de Google et de PostHog : le navigateur en a besoin pour initialiser le service, le serveur pour vérifier ce qui lui revient.

Ce que fait Yamba quand une variable manque

La plateforme est conçue pour démarrer sans presque rien. Chaque brique optionnelle se désactive proprement plutôt que de faire échouer le démarrage.

Manque	Conséquence
Clé Stripe	Le fournisseur de paiement factice prend le relais : les réservations fonctionnent, aucun argent ne bouge. Refusé en production
Configuration email	Les emails sont écrits en mémoire et jamais envoyés. Refusé en production
Redpanda	Le relais d'événements et les consommateurs restent en attente, l'API vit normalement
Clé PostHog	Aucune bannière de consentement, aucun envoi, aucune erreur
Adresse Sentry	Aucune remontée d'erreur, aucun bruit
Identifiant Google	Le bouton affiche « bientôt disponible »
Clés de chiffrement	Hors production, une clé de développement est dérivée avec un avertissement. En production, le démarrage est refusé

2. Démarrage local, de zéro à une plateforme qui tourne

Le strict minimum

Quatre variables suffisent à faire tourner l'ensemble en développement.

```
DATABASE_URL="mongodb+srv://..." # MongoDB Atlas, avec replica set
REDIS_DATABASE_URI="redis://..." # Redis, local ou hébergé
ACCESS_TOKEN_SECRET="..." # openssl rand -base64 48
REFRESH_TOKEN_SECRET="..." # openssl rand -base64 48
```

La séquence complète

```
npm ci # après tout changement d'outillage
docker compose up -d # Redpanda et Mailpit
./scripts/redpanda-bootstrap.sh # une seule fois : crée les sujets d'événements
npx prisma generate
npx prisma db push # synchronise le schéma et les index
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-deals.ts # jeu d'essai
npx tsx --env-file=.env packages/libs/prisma/scripts/grant-admin.ts votre@email --role SUPER_ADMIN
npm run dev # les six services et le site
npx nx dev admin-ui # le back-office, dans un autre terminal
bash scripts/smoke-services.sh # les six démarrent-ils vraiment ?
```

Attention aux scripts : lancez-les toujours avec `npx tsx --env-file=.env`, jamais en sourçant le `.env` dans le terminal. Le mot de passe MongoDB contient des caractères que le shell interprète.

Tester depuis un téléphone sur le réseau local

Deux réglages dans `apps/user-ui/.env.local`, puis redémarrage :

```
API_PROXY_TARGET=http://localhost:8080
NEXT_PUBLIC_API_BASE_URL=/api
```

Le site relaie alors les appels vers la passerelle, et les cookies restent valides quel que soit l'hôte. Sans cela, une connexion réussit puis la session est perdue immédiatement, parce qu'un cookie est lié à son domaine.

3. Tableau complet des variables

Indispensables

Variable	Lu par	Rôle	Sans elle
DATABASE_URL	Tous	MongoDB Atlas. Le replica set est obligatoire : les transactions en dépendent	Rien ne fonctionne
REDIS_DATABASE_URI	Tous	Sessions, verrous, caches, battements de tâches	Sessions et verrous cassés
ACCESS_TOKEN_SECRET	auth, gateway	Signature des jetons de session	Aucune connexion
REFRESH_TOKEN_SECRET	auth	Signature des jetons de renouvellement	Aucune session durable

Paie ment

Variable	Rôle	Sans elle
STRIPE_SECRET_KEY	Clé secrète, autorisation puis capture	Fournisseur factice, refusé en production
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Formulaire de carte côté navigateur	Pas de saisie de carte
STRIPE_WEBHOOK_SECRET	Signature du webhook principal	Le webhook répond 501
STRIPE_CONNECT_WEBHOOK_SECRET	Signature du second webhook, comptes connectés	Versements en échec non remontés

Email

Variable	Rôle	Sans elle
EMAIL_PROVIDER	resend , smtp ou fake . Déduit si absent	Déduction automatique
RESEND_API_KEY	Envoi par Resend	Repli sur SMTP
RESEND_WEBHOOK_SECRET	Signature des retours Resend	Rebonds et plaintes non traités
SMTP_HOST , SMTP_PORT , SMTP_USER , SMTP_PASS	Envoi par SMTP	Repli sur le fournisseur factice
EMAIL_FROM , SMTP_FROM , SMTP_FROM_NAME	Expéditeur affiché	Valeur par défaut
SUPPORT_EMAIL	Adresse rappelée dans les emails et destinataire des alertes. Défaut support@yamba.app	Défaut utilisé

Images

Variable	Rôle	Sans elle
IMAGEKIT_PUBLIC_KEY , IMAGEKIT_PRIVATE_KEY , IMAGEKIT_URL_ENDPOINT	Signature des téléversements côté serveur, suppression de fichiers	Tout téléversement échoue : photos de colis, justificatifs, avatars
NEXT_PUBLIC_IMAGEKIT_PUBLIC_KEY , NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT	Téléversement depuis le navigateur	Idem

Chiffrement

Variable	Rôle	Sans elle
DELIVERY_CODE_ENCRYPTION_KEY	Chiffre le code de livraison pour le réafficher à l'Expéditeur	Clé de développement hors production, démarrage refusé en production
TOTP_ENCRYPTION_KEY	Chiffre le secret de double authentification des administrateurs	Idem

Connexion Google et cartes

Variable	Rôle	Sans elle
GOOGLE_CLIENT_ID	Le serveur vérifie que le jeton lui est bien destiné	Connexion Google refusée
NEXT_PUBLIC_GOOGLE_CLIENT_ID	Le navigateur initialise le bouton	Bouton « bientôt disponible »
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY	Saisie assistée des villes	Pas d'autocomplétion

Back-office

Variable	Rôle	Défaut
TOTP_ENCRYPTION_KEY	Voir chiffrement	—
ADMIN_TOTP_ISSUER	Nom affiché dans l'application d'authentification	Yamba Admin
ADMIN_SESSION_INACTIVITY_MINUTES	Inactivité tolérée	45
ADMIN_SESSION_LIFETIME_HOURS	Durée de vie absolue	12
ADMIN_PREAUTH_MINUTES	Délai entre le mot de passe et le code	5
ADMIN_UI_URL	Adresse publique du back-office, liens d'invitation	http://localhost:3001

Événements et tâches planifiées

Variable	Rôle	Défaut
KAFKA_BROKERS	Adresse de Redpanda	localhost:9092
OUTBOX_RELAY_ENABLED , MESSAGING_RELAY_ENABLED	Relais d'événements	actifs
NOTIFICATION_CONSUMER_ENABLED	Consommateur de notifications	actif
BOOKING_EXPIRY_CRON_ENABLED	Expiration des demandes à 24 h	actif
BOOKING_PAYOUT_CRON_ENABLED	Versement à J+4 et rejeu	actif
OPS_DIGEST_CRON_ENABLED	Récapitulatif quotidien au support	actif
OPS_ALERTS_CRON_ENABLED	Alertes de seuil horaires	actif
RATING_CRON_ENABLED	Relances de notation	actif
RECIPIENT_REDACTION_CRON_ENABLED	Effacement du destinataire	actif
OUTBOX_RETENTION_CRON_ENABLED , RETENTION_CRON_ENABLED	Purges de conservation	actifs
MESSAGING_REMINDER_CRON_ENABLED , MESSAGING_RETENTION_CRON_ENABLED	Relance des non-lus, purge des fils	actifs
ONBOARDING_REMINDER_CRON_ENABLED	Rappels d'inscription Voyageur	actif
CRON_HEARTBEAT_PING_URLS	Battements vers le moniteur externe, au format {"service:tâche": "url"}	aucun envoi

Mettre `false` désactive. Utile pour faire tourner plusieurs instances de l'API sans dupliquer les tâches.

Observabilité et maintenance

Variable	Rôle
<code>SENTRY_DSN</code> , <code>SENTRY_ENVIRONMENT</code>	Remontée des erreurs serveur
<code>NEXT_PUBLIC_SENTRY_DSN</code>	Remontée côté navigateur
<code>NEXT_PUBLIC_POSTHOG_KEY</code> , <code>NEXT_PUBLIC_POSTHOG_HOST</code>	Mesure d'audience, après consentement
<code>POSTHOG_API_KEY</code> , <code>POSTHOG_HOST</code>	Événements serveur, membres consentants uniquement
<code>MAINTENANCE_MODE</code>	<code>on</code> force la lecture seule, même base injoignable
<code>MAINTENANCE_MESSAGE_FR</code> , <code>MAINTENANCE_MESSAGE_EN</code>	Message affiché
<code>GATEWAY_URL</code> , <code>APP_VERSION</code> , <code>GIT_SHA</code>	Page d'état et version affichée

Front

Variable	Rôle
<code>NEXT_PUBLIC_API_BASE_URL</code>	Adresse de l'API. <code>/api</code> avec le relais, sinon <code>http://localhost:8080/api</code>
<code>API_PROXY_TARGET</code>	Relais des appels par le site, indispensable en réseau local
<code>USER_APP_URL</code>	Adresse publique du site, utilisée dans les emails. Défaut <code>http://localhost:3000</code>
<code>NEXT_PUBLIC_APP_ENV</code>	Bandeau d'environnement
<code>CORS_ORIGIN</code>	Origines autorisées côté services

4. Les services externes, un par un

MongoDB Atlas

Créez un cluster. **Un replica set est obligatoire** : Yamba écrit ses événements dans la même transaction que le changement d'état, et Mongo n'offre les transactions qu'en replica set. Un cluster partagé gratuit convient au développement, mais il plafonne les agrégations à cinquante étapes, ce qui a déjà causé des erreurs sur les mises à jour larges.

Dans `Network Access` , autorisez votre adresse. Dans `Database Access` , créez un utilisateur. Copiez la chaîne de connexion dans `DATABASE_URL` , sans oublier le nom de la base à la fin.

Pour la production, prenez un palier qui offre les sauvegardes continues, et faites une restauration d'essai avant le lancement.

Redis

Une instance locale suffit en développement. Renseignez `REDIS_DATABASE_URI`. Redis porte les sessions, les verrous de tâches, les caches et les battements : sans lui, les connexions ne tiennent pas.

Redpanda

Fourni par le fichier `docker-compose.yml`. Lancez `docker compose up -d` puis, une seule fois, `./scripts/redpanda-bootstrap.sh` pour créer les sujets. Sans Redpanda, l'API fonctionne mais les emails et notifications déclenchés par un événement n'arrivent pas.

Stripe

Dans le tableau de bord, en mode test, relevez la clé secrète et la clé publiable. Activez Connect, en comptes Express : c'est ce qui permet de reverser aux Voyageurs.

Pour les webhooks en développement, utilisez l'outil en ligne de commande :

```
stripe listen --forward-to localhost:6003/webhooks/stripe
```

Il affiche un secret de signature à mettre dans `STRIPE_WEBHOOK_SECRET`. En production, créez **deux** points de terminaison sur la même adresse : un pour les événements de compte, un second en cochant « événements des comptes connectés », dont le secret va dans `STRIPE_CONNECT_WEBHOOK_SECRET`.

Sans clé secrète, le fournisseur factice prend le relais et les parcours fonctionnent de bout en bout sans argent réel. C'est le mode conseillé pour la recette.

ImageKit

Créez un compte, relevez dans le tableau de bord la clé publique, la clé privée et le point d'entrée d'URL. Renseignez les trois côté serveur, et les deux publiques côté site.

Le navigateur téléverse directement chez ImageKit, après avoir demandé une signature au serveur. C'est pour cela que la clé privée reste côté serveur : elle signe, elle ne circule jamais.

Google Maps

Dans la console Google Cloud, activez `Places API` et `Maps JavaScript API`, créez une clé, restreignez-la à vos domaines. Elle sert à la saisie assistée des villes.

Connexion Google

Dans la console Google Cloud, configurez d'abord l'écran de consentement : type externe, nom de l'application, email de contact, portées `email`, `profile`, `openid`. Tant que l'application est en test, seuls les comptes déclarés comme testeurs peuvent se connecter, ce qui suffit pour la recette.

Créez ensuite un identifiant de type **ID client OAuth, application Web**. Renseignez les origines JavaScript autorisées, sans chemin ni barre oblique finale :

```
http://localhost:3000
http://192.168.1.155:3000
```

Laissez les URI de redirection vides : Yamba utilise Google Identity Services, qui renvoie un jeton au navigateur sans redirection.

Reportez le même identifiant dans les deux fichiers :

```
# .env
GOOGLE_CLIENT_ID=123456789-xxxx.apps.googleusercontent.com
# apps/user-ui/.env.local
NEXT_PUBLIC_GOOGLE_CLIENT_ID=123456789-xxxx.apps.googleusercontent.com
```

Le serveur vérifie que le jeton reçu a bien été émis pour cet identifiant. C'est ce contrôle qui empêche de présenter un jeton Google obtenu ailleurs.

Le piège le plus courant est une origine mal saisie : Google refuse alors le chargement avec une erreur dans la console du navigateur. Si vous testez depuis un téléphone, c'est l'adresse réseau qu'il faut avoir déclarée, pas `localhost`.

PostHog

Créez un compte sur <https://eu.posthog.com> en choisissant la région **Union européenne** : les données doivent rester en Europe, et la région ne se change pas ensuite. Créez un projet, puis relevez la **Project API Key**, qui commence par `phc_`.

La même clé sert au navigateur et au serveur. Elle est publique par nature : c'est une clé d'envoi, pas une clé de lecture.

```
# apps/user-ui/.env.local
NEXT_PUBLIC_POSTHOG_KEY=phc_votre_cle
NEXT_PUBLIC_POSTHOG_HOST=https://eu.i.posthog.com
# .env
POSTHOG_API_KEY=phc_votre_cle
POSTHOG_HOST=https://eu.i.posthog.com
```

Reconstruisez le front. Rien ne part tant que la personne n'a pas accepté la bannière : refuser signifie qu'aucun script n'est chargé, pas seulement qu'aucun événement n'est envoyé. Côté serveur, seuls les membres ayant coché la mesure d'audience produisent des événements, et les propriétés passent par une liste blanche : ni destinataire, ni email, ni téléphone, ni code de livraison ne peuvent sortir.

Resend

Créez un compte, ajoutez votre domaine et publiez les enregistrements DNS demandés. Relevez une clé d'API. Ajoutez un webhook vers `/webhooks/email/resend` de notification-service et relevez son secret de signature.

```
EMAIL_PROVIDER=resend
RESEND_API_KEY=re_XXX
RESEND_WEBHOOK_SECRET=whsec_XXX
EMAIL_FROM="Yamba <bonjour@votre-domaine>"
```

En développement, préférez Mailpit : `EMAIL_PROVIDER=smtp` , `SMTP_HOST=localhost` , `SMTP_PORT=1025` , et vous lisez les emails sur `http://localhost:8025` .

Sentry

Créez un projet Node pour les services et un projet Next pour le site. Renseignez `SENTRY_DSN` et `NEXT_PUBLIC_SENTRY_DSN` . Sans adresse, la capture est inerte, sans erreur ni bruit.

Moniteur externe

Créez un compte Better Stack, offre gratuite. Trois surveillances HTTP : la sonde publique de la passerelle, et la santé des deux fronts. Quatre battements pour les tâches les plus critiques, dont les adresses vont dans `CRON_HEARTBEAT_PING_URLS` . Le détail figure dans la documentation technique.

5. Générer les secrets

```
openssl rand -base64 48      # ACCESS_TOKEN_SECRET, REFRESH_TOKEN_SECRET
openssl rand -base64 32      # DELIVERY_CODE_ENCRYPTION_KEY, TOTP_ENCRYPTION_KEY
```

Les deux clés de chiffrement font exactement trente-deux octets : une valeur d'une autre longueur est refusée au démarrage. Ne les changez jamais sur une base existante, sinon les codes de livraison et les secrets de double authentification déjà enregistrés deviennent illisibles.

Aucun fichier `.env` ne doit être versionné. Un contrôle d'intégration continue refuse tout fichier sensible ajouté par mégarde.

6. Vérifier que tout fonctionne

```
bash scripts/smoke-services.sh      # les six services démarrent-ils ?
curl -s localhost:8080/api/status | head -c 200  # sonde publique : ok, degraded, down
curl -s localhost:6001/health        # santé d'un service : base et cache
curl -s localhost:3000/api/health    # santé du site
```

Le back-office offre une page « État des services » qui affiche la même chose, plus le dernier passage de chaque tâche planifiée, le retard des événements et les emails en échec.

Symptôme	Cause la plus fréquente
Connexion réussie puis session perdue	Front et API sur des hôtes différents : utilisez le relais
Aucun email reçu	Aucun fournisseur configuré : les envois sont en mémoire

Symptôme	Cause la plus fréquente
Téléversement de photo en échec	Clés ImageKit absentes du <code>.env</code> de la racine
Le téléversement marche mais la suppression non	Clés posées dans <code>apps/trip-service/.env</code> seulement : auth-service ne les voit pas
Bouton Google inerte	Identifiant public absent, ou origine non déclarée
Pas de bannière de consentement	Clé PostHog absente
Notification jamais reçue	Redpanda arrêté, ou sujets non créés
Démarrage refusé en production	Une clé de chiffrement manque

7. État actuel de votre installation

Constaté le 06/09/2026 sur le poste de développement.

Présentes : base de données, cache, les deux secrets de jetons, la clé Stripe secrète, la configuration SMTP, la clé Google Maps, l'adresse du site.

Absentes, avec leur conséquence :

Manque	Conséquence
~~Les trois clés ImageKit~~	Réglé le 06/09 : elles vivaient dans <code>apps/trip-service/.env</code> et ont été recopiées à la racine. Chaîne vérifiée de bout en bout — signature du serveur, téléversement réel, image servie, suppression effective
<code>DELIVERY_CODE_ENCRYPTION_KEY</code> , <code>TOTP_ENCRYPTION_KEY</code>	Clés de développement dérivées, avec avertissement. Bloquant pour la production
<code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code> , les deux secrets de webhook	Pas de saisie de carte, webhooks inactifs
Identifiant Google	Bouton « bientôt disponible »
Clés PostHog	Aucune mesure d'audience, huit scénarios de recette non testables
<code>SENTRY_DSN</code>	Aucune remontée d'erreur
<code>SUPPORT_EMAIL</code> , <code>ADMIN_UI_URL</code>	Valeurs par défaut, suffisantes en développement

Ces trois variables ImageKit ne figuraient pas non plus dans `.env.example` alors que le code les lit : elles y ont été ajoutées.

Vestiges à supprimer : `apps/trip-service/.env` et `apps/user-ui/.env` datent de mai et font désormais doublon avec la racine. Deux sources pour une même valeur finissent toujours par diverger.

8. Avant la production

- Les deux clés de chiffrement sont posées et sauvegardées ailleurs que sur le serveur.

- Le fournisseur d'email est `resend` ou `smtp` : le mode factice est refusé.
- Le fournisseur de paiement est Stripe, en clés de production, avec les deux webhooks.
- `NODE_ENV=production` sur tous les services.
- Le cluster de base offre les sauvegardes, et une restauration d'essai a été faite.
- Le moniteur externe est actif et a déjà envoyé une alerte de test.
- Les origines autorisées ne contiennent plus `localhost` ni d'adresse de réseau local.
- Chaque secret est stocké dans un gestionnaire de secrets, pas dans un fichier sur la machine.

Le premier démarrage en production activera les rappels d'inscription : vérifiez le volume attendu, ou laissez la tâche coupée le temps d'un premier passage.

9. Préparer le poste pour le chantier mobile (D36, D73)

Ce chapitre a son propre document, plus complet : `06-YAMBA-PREPARATION-MOBILE.md`. Il couvre les outils à installer avec leurs commandes, ce qui est déjà présent sur le poste, ce qui est gratuit ou non pour tester sur un vrai téléphone, le blocage du dépôt App Store depuis ce Mac et son contournement, le choix entre compte individuel et compte organisation, l'usage du compte d'un tiers, et le calendrier des dépenses.